Tritium decay with Euler's method

Python | Thu 15 Oct 2026 | ~25 min

Tritium, the fuel stored next to a D-T reactor, decays with a half-life of 12.32 years. The number of nuclei follows

$$\frac{dN}{dt} = -\lambda N, \qquad \lambda = \frac{\ln 2}{t_{1/2}}$$

You'll solve this ODE with the explicit Euler method and measure how the error depends on the time step.

New to Euler's method? Read the Theory pages first.

Tasks

Fill in `solution.py`. Use only `numpy` and `math`, not `scipy`.

1. `decay_rate(half_life)`: return λ.
2. `euler_decay(N0, lam, dt, t_end)`: integrate from t = 0 to t_end with explicit Euler, using a loop. Return two NumPy arrays `t` and `N` that include both the start and the end point. You can assume `t_end / dt` is a whole number.
3. `exact_decay(N0, lam, t)`: return the analytic solution at times `t` (works for a number or an array).
4. `main()`: take N0 = 1e6, t_end = 50 years, and dt = 5, 2.5, 1.25, 0.625 years. For each dt, print:
   - dt
   - the relative error at `t_end`, $|N_{\text{euler}} - N_{\text{exact}}| \,/\, N_{\text{exact}}$
   - the ratio of the previous error to this one (skip it for the first row)

Check yourself

Your relative error for dt = 5 should be about 0.39. Look at the ratio column and add one comment line at the bottom of `solution.py` saying what it tells you about Euler's method.

Submit

Commit and push `solution.py`.

THEORY

Euler's method

The idea

An ODE tells you the slope of the solution at every point:

$$\frac{dy}{dt} = f(t, y), \qquad y(t_0) = y_0$$

Euler's method walks a short step Δt in the direction of the current slope, then works out the slope again at the new point, and repeats:

$$y_{n+1} = y_n + \Delta t\, f(t_n, y_n), \qquad t_{n+1} = t_n + \Delta t$$

Here y_n is the approximation of $y(t_n)$.

For decay, $f = -\lambda N$, so one step is

$$N_{n+1} = N_n - \lambda N_n\, \Delta t$$

Where it comes from

Taylor-expand the exact solution:

$$y(t + \Delta t) = y(t) + \Delta t\, y'(t) + \tfrac{1}{2}\Delta t^2\, y''(t) + \dots$$

Euler keeps the first two terms and drops the rest. Each step therefore makes an error of order Δt^2. This is the *local error*.

Error and order

To reach a fixed time T you take $T/\Delta t$ steps, and the local errors add up: $(T/\Delta t) \cdot \Delta t^2 \sim \Delta t$.

In general a method's *global error* at a fixed time behaves like

$$E \approx C\, \Delta t^{\,p}$$

where p is the **order** of the method. Halving Δt divides the error by 2^p. The ratio of successive errors therefore tells you p, and that ratio is what the last column of your table measures.

Stability (worth knowing)

For decay, each Euler step multiplies N by $(1 - \lambda\Delta t)$:

- If $\lambda\Delta t > 1$, the factor is negative, so N flips sign every step.
- If $\lambda\Delta t > 2$, $|1 - \lambda\Delta t| > 1$, so N grows without bound.

The exact solution never does either, so the step size is limited by stability as well as accuracy. In this exercise $\lambda\Delta t \le 0.28$, which is safe.

Practical tips

- **Number of steps:** use `n = round(t_end / dt)`, not `int(...)`. Floating-point division can land just below a whole number: `int(0.3 / 0.1)` gives `2`, not `3`.
- **Preallocate:** create `N = np.zeros(n + 1)`, then fill it in the loop. Fortran and MATLAB work the same way.